Temat: Zapytania zagnieżdżone, złączenia, widoki

1. Zapytanie zagnieżdżone (podzapytanie)

> *Zapytanie zagnieżdżone (podzapytanie) jest typem zapytania użytym wewnątrz instrukcji SELECT (innego zapytania). Serwer baz danych wykona najpierw zapytanie wewnętrzne (podzapytanie), aby jego wynik wykorzystać do wykonania zapytania zewnętrznego. Może być stosowanych wiele poziomów zagnieżdżenia zapytań. W podzapytaniach mogą być używane konstrukcje EXISTS, NOT EXISTS, IN, NOT IN wraz z klauzulą WHERE oraz operatorami.*

```
SELECT atrybuty FROM tabela WHERE kolumna operator ( SELECT... );
```

Schemat ogólny: Zewnętrzna kwerenda filtruje wiersze (WHERE) wykorzystując wynik zwrócony przez zapytanie wewnętrzne.

```
SELECT imie, pensja FROM pracownicy WHERE pensja > ( SELECT AVG(pensja) FROM pracownicy );
```

Działanie: System najpierw oblicza średnią pensję w firmie, a następnie zwraca dane pracowników zarabiających więcej niż ta średnia.

```
SELECT nazwa FROM produkty WHERE id_klienta IN ( SELECT id FROM kategorie WHERE nazwa='Komputery' );
```

Działanie: Podzapytanie znajduje ID kategorii 'Komputery', a zapytanie zewnętrzne wybiera produkty należące do tych kategorii.

```
SELECT * FROM klienci k WHERE EXISTS (SELECT * FROM zamowienia z WHERE z.id_klienta = k.id);
```

Działanie: Konstrukcja `EXISTS` sprawdza istnienie powiązania. Zwraca klientów, którzy złożyli przynajmniej jedno zamówienie.

2. Zapytanie skorelowane

> *Zapytanie skorelowane – zapytanie, w którym zapytanie wewnętrzne odwołuje się do wartości z aktualnie przetwarzanego wiersza zapytania zewnętrznego.*

```
SELECT imie, nazwisko FROM pracownicy p WHERE pensja > ( SELECT AVG(pensja) FROM pracownicy WHERE dzial=p.dzial );
```

Działanie: Podzapytanie liczy średnią pensję osobno dla każdego działu pracownika (`dzial = p.dzial`). Zwraca osoby zarabiające powyżej średniej w swoim dziale.

3. Złączenia (JOIN)

> *Złączenia (JOIN) są używane, gdy zachodzi konieczność uzyskania dostępu do więcej niż jednej tabeli lub więcej niż jednej krotki. Złączenia mogą być:*
> - *wewnętrzne (INNER JOIN)*
> - *zewnętrzne (OUTER JOIN)*
> - *złączenie krzyżowe (CROSS JOIN)*
> - *złączenie w obrębie jednej tabeli (SELF JOIN)*

Tabele bazowe (wykorzystywane w przykładach)

Table: Customers

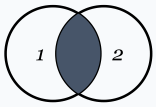
customer_id	first_name
1	John
2	Robert
3	David
4	John
5	Betty

Table: Orders

order_id	amount	customer_id
1	200	10
2	500	3
3	300	6
4	800	5
5	150	8

a) Złączenia wewnętrzne - INNER JOIN

> *Tabela wynikowa będzie zawierać tylko takie elementy, dla których zostaną spełnione warunki złączenia wewnętrznego. Mogą to być:*
> - *równozłączenia*
> - *nierównozłączenia (<, >, !=, BETWEEN)*



INNER JOIN - Przykład i Wynik

```
SELECT Customers.customer_id, Customers.first_name, Orders.amount FROM Customers
INNER JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

Działanie: Zwraca tylko te rekordy, które mają swoje odpowiedniki w obu tabelach (ID 3 oraz 5).

Wynik złączenia INNER JOIN:

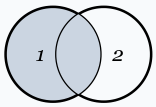
customer_id	first_name	amount
3	David	500
5	Betty	800

b) Złączenia zewnętrzne - OUTER JOIN

> *Złączenia zewnętrzne (OUTER JOIN) - zwracają również krotki niepasujące do warunku. Wyróżnia się 3 typy: LEFT, RIGHT, FULL OUTER JOIN.*

LEFT OUTER JOIN

> *LEFT OUTER JOIN - zawiera wszystkie krotki pierwszej tabeli oraz pasujące z drugiej.*




```
SELECT Customers.customer_id, Customers.first_name, Orders.amount FROM Customers
LEFT JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

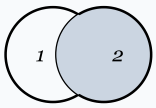
Działanie: Wypisuje wszystkich klientów. Jeśli brak zamówienia, w kolumnie amount pojawia się NULL.

Wynik złączenia LEFT JOIN:

customer_id	first_name	amount
1	John	*NULL*
2	Robert	*NULL*
3	David	500
4	John	*NULL*
5	Betty	800

RIGHT OUTER JOIN

> *RIGHT OUTER JOIN - zawiera wszystkie krotki drugiej tabeli oraz pasujące z pierwszej.*




```
SELECT Customers.customer_id, Customers.first_name, Orders.amount FROM Customers
RIGHT JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

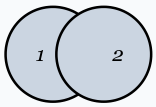
Działanie: Wypisuje wszystkie zamówienia. Jeśli klient nie istnieje w Customers, w kolumnach klienta pojawia się NULL.

Wynik złączenia RIGHT JOIN:

customer_id	first_name	amount
3	David	500
5	Betty	800
NULL	*NULL*	200
NULL	*NULL*	300
NULL	*NULL*	150

FULL OUTER JOIN

> *FULL OUTER JOIN - zawiera wszystkie krotki obu złączonych tabel (również te bez odpowiedników).*




```
SELECT Customers.customer_id, Customers.first_name, Orders.amount FROM Customers
FULL OUTER JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

Działanie: Łączy zachowanie LEFT i RIGHT JOIN. Wypisuje pełen zestaw rekordów z obu stron.

Wynik złączenia FULL OUTER JOIN:

customer_id	first_name	amount
1	John	*NULL*
2	Robert	*NULL*
3	David	500
4	John	*NULL*
5	Betty	800
NULL	*NULL*	200
NULL	*NULL*	300
NULL	*NULL*	150

c) Pozostałe rodzaje złączeń

> ***CROSS JOIN*** *- otrzymuje się kombinację każdej krotki pierwszej tabeli z każdą krotką drugiej tabeli (iloczyn kartezjański).*

```
SELECT Customers.customer_id, Customers.first_name, Orders.order_id FROM Customers
CROSS JOIN Orders;
```

Działanie: Łączy każdego klienta z każdym zamówieniem (5 * 5 = 25 wierszy).

Wynik złączenia CROSS JOIN (przykładowe wiersze):

customer_id	first_name	order_id
1	John	1
1	John	2
2	Robert	1
2	Robert	2

> ***SELF JOIN*** *- złączenie w obrębie jednej tabeli, gdy po obu stronach występuje ta sama tabela (używa się aliasów).*

Tabela pracownicy (dla SELF JOIN)

id	imie	id_kierownika
1	Adam	*NULL*
2	Jan	1
3	Anna	1
4	Basia	2

```
SELECT p.imie AS pracownik, k.imie AS kierownik FROM pracownicy p LEFT JOIN pracownicy k ON p.id_kierownika = k.id;
```

Działanie: Odczytuje tę samą tabelę podwójnie (jako pracownicy i kierownicy), pozwalając ustalić relacje zwierzchnictwa.

Wynik złączenia SELF JOIN:

pracownik	kierownik
Adam	*NULL*
Jan	Adam
Anna	Adam
Basia	Jan

d) Wiele tabel...

```
SELECT c.first_name, o.item, s.status FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id LEFT JOIN Shippings s ON o.customer_id = s.customer_id;
```

Działanie: Łączenie więcej niż dwóch tabel sekwencyjnie za pomocą kolejnych klauzul JOIN.

4. Widoki (perspektywy)

> *Widoki (perspektywy) - wirtualne tabele, których zawartość jest wyliczana przez system z kilku tabel źródłowych. Perspektywa jest definiowana jako zapytanie (kwerenda).*
> - *Tabela - przechowuje dane*
> - *Widok - przechowuje definicję zapytania*

```
CREATE VIEW produkty_tanie AS SELECT nazwa, cena FROM produkty WHERE cena < 100;
```

Działanie: Tworzy wirtualną tabelę (widok) przechowującą jedynie zapytanie filtrujące tanie produkty.

```
CREATE VIEW historia_zamowien AS SELECT k.nazwa, z.id, z.data FROM klienci k INNER JOIN zamowienia z ON k.id = z.id_klienta;
```

Działanie: Tworzy widok łączący klientów z zamówieniami.

```
SELECT * FROM historia_zamowien;
```

Działanie: Odpytanie widoku tak, jak zwykłej tabeli.

> *Widoki mogą zawierać funkcje agregujące (np. AVG()), grupowanie (GROUP BY), mogą też zawierać zapytania zagnieżdżone.*

```
CREATE OR REPLACE VIEW pracownicy_it AS SELECT...
```

Działanie: Modyfikacja (nadpisanie) istniejącego widoku.

```
DROP VIEW nazwa_widoku;
```

Działanie: Usunięcie widoku z bazy danych.

Zalety widoków:

- Upraszczanie zapytań
- Bezpieczeństwo
- Łatwiejsze zarządzanie
- Ponowne wykorzystanie kodu

Wady widoków:

- Mogą obniżać wydajność
- Ograniczona możliwość modyfikacji